

目錄

1. [JPA / Hibernate 進階映射](#) 概念：說明為什麼需要「進階映射」——真實資料庫不會只有一張表，而是多張表彼此關聯，單一映射（一個 Java Class 對一張表）不夠用。
2. [一對一映射 \(One-to-One Mapping\)](#) 概念：用「講師 (Instructor)」與「講師詳細資料 (Instructor Detail)」為例，說明一個實體對應另一個實體的關係，並附上 ER 圖。
3. [一對多映射 \(One-to-Many Mapping\)](#) 概念：一名講師可以開設多門課程，說明一個實體對應多個實體的關係。
4. [多對一映射 \(Many-to-One Mapping\)](#) 概念：一對多的反向角度——多門課程指向同一位講師。
5. [多對多映射 \(Many-to-Many Mapping\)](#) 概念：用學生與課程為例，說明兩邊都可以有多筆對應關係的複雜關聯。
6. [資料庫關聯性基礎概念](#) 概念：複習主鍵、外鍵、級聯操作，這三個是深入 JPA 映射前必懂的資料庫基礎機制。
7. [資料庫關聯性核心要素](#) 概念：進一步定義主鍵（唯一識別一列資料）與外鍵（指向另一表主鍵）的作用。
8. [外鍵的實務應用範例](#) 概念：用 instructor_detail_id 這個欄位實際示範外鍵怎麼把兩張表串起來。
9. [資料庫級聯操作 \(Cascading\)](#) 概念：定義級聯——對主實體做的操作（如儲存、刪除）自動套用到關聯實體上，並用刪除講師連帶刪除其詳細資料為例。
10. [刪除級聯 \(Cascading Delete\) 的運作實例](#) 概念：具體示範刪除 Darby 講師時，系統會自動刪除他的 instructor_detail 資料，並提醒級聯刪除要依業務情境謹慎使用。
11. [級聯操作的決策邏輯](#) 概念：用「刪除學生不該連帶刪除課程」這個反例，說明級聯設定要看業務邏輯決定，不是為了方便就全部打開。
12. [資料檢索策略預覽](#) 概念：預告接下來要講的兩種資料讀取模式——及時載入與延遲載入。
13. [及時載入 \(Eager Loading\) 與 延遲載入 \(Lazy Loading\) 詳解](#) 概念：說明查詢時是要一次把關聯資料都抓回來 (Eager)，還是等真的用到才去資料庫拿 (Lazy)。
14. [單向關聯 \(Unidirectional Relationship\) 概念預覽](#) 概念：用 Instructor → Instructor Detail 為例，說明關聯只能單方向查詢（反過來不行）。
15. [單向與雙向關聯 \(Unidirectional vs. Bidirectional\)](#) 概念：比較單向（只能從一邊查另一邊）跟雙向（兩邊都能互查）關聯的差別。
16. [資料建模的靈活性與適應性](#) 概念：強調一對一 / 一對多 / 多對一 / 多對多都只是通用範例，實際設計要依需求調整，沒有標準答案。
17. [Hibernate 一對一映射 \(One-to-One Mapping\)](#) 概念：正式進入本篇主軸——用講師與其詳細資料的一對一關聯，從資料庫到 Java 程式碼一步步實作。

18. **單向一對一關聯 (Unidirectional One-to-One)** 概念：說明這次要做的是「只能從講師查到詳細資料」的單向版本，是學雙向關聯前的暖身。
19. **一對一映射 (One-to-One Mapping) 的開發流程** 概念：列出三個開發步驟：先準備資料庫、再寫 Entity 類別、最後整合到應用程式。
20. **初始資料庫設計：Instructor Detail 表** 概念：開始設計 instructor_detail 表的欄位，先從主鍵 id 開始。
21. **實作 instructor_detail 資料表腳本** 概念：寫出建立 instructor_detail 表的 SQL 腳本，包含自動遞增主鍵、頻道名稱、興趣欄位。
22. **instructor 資料表結構初步規劃** 概念：初步列出 instructor 表要有哪些欄位（先列 id、first_name）。
23. **instructor 資料表完整結構規劃** 概念：補齊 instructor 表完整欄位，並加入 instructor_detail_id 這個之後要當外鍵用的欄位，但此時兩表還沒有正式關聯。
24. **外鍵 (Foreign Key) 的概念與作用** 概念：定義外鍵——一個表裡引用另一個表主鍵的欄位，用來把兩張表連結起來。
25. **實作一對一關聯：以 Instructor 為例** 概念：用流程圖示範外鍵怎麼從 instructor 表指向 instructor_detail 表的主鍵。
26. **在 SQL 中實作外鍵約束 (Foreign Key Constraint)** 概念：用 CONSTRAINT ... FOREIGN KEY 語法把外鍵規則正式寫進資料表建立腳本。
27. **參照完整性 (Referential Integrity)** 概念：解釋定義外鍵的真正目的是確保「如果 A 引用了 B，B 就一定要存在」這條規則。
28. **外鍵約束的防禦機制** 概念：說明外鍵約束會擋掉無效的外鍵值，插入不存在的主鍵會直接報錯。
29. **實作進度總結與展望** 概念：小結目前完成的資料庫端工作，準備轉往 Java 程式碼層級的實作。
30. **進入 Java 實體類別開發階段** 概念：正式開始寫 InstructorDetail 這個 Java 類別，對應到資料庫的 instructor_detail 表。
31. **實作 InstructorDetail 實體類別映射** 概念：用 @Entity、@Table 註解把 InstructorDetail 類別對應到資料表，並映射主鍵與一般欄位。
32. **實作 Instructor 實體類別映射** 概念：比照辦理，用 @Entity、@Table 建立 Instructor 類別對應 instructor 表。
33. **建立實體類別間的關聯映射** 概念：指出兩個類別目前雖然都各自映射好了，但彼此還沒有關聯，需要用 @OneToOne 把它們串起來。
34. **實作關聯的連結機制 (Hooking up the Relationship)** 概念：用 @JoinColumn 指定實際存放外鍵的欄位，讓 Hibernate 知道怎麼去對應表撈資料。
35. **Hibernate 實體生命週期 (Entity Life Cycle)** 概念：介紹 Hibernate 實體在應用程式執行期間會經歷的狀態，先從「游離狀態 (Detached)」開始。
36. **Hibernate 實體狀態操作方法** 概念：介紹 merge、persist、remove、refresh 四個操作方法分別用來處理實體的哪種狀態轉換。

37. **Hibernate 實體狀態轉換流程圖** 概念：用狀態圖整理 Transient、Persistent、Detached 三種狀態之間怎麼轉換。
38. **狀態與操作細節說明** 概念：細部說明每個狀態（暫時、持久化、游離）的定義跟能做的操作。
39. **Hibernate 狀態轉換的進階細節** 概念：補充 remove 之後會進入 Removed 狀態、rollback 會退回 Detached 狀態等細節，並提醒不用死背所有轉換路徑。
40. **Hibernate 級聯操作 (Cascading)** 概念：再次定義級聯——對主實體的操作自動套用到關聯實體，減少手動處理多個物件的麻煩。
41. **Hibernate 級聯操作類型 (Cascade Types)** 概念：說明級聯不是全開全關的開關，而是可以用 CascadeType 精確選擇要套用到哪些操作。
42. **級聯刪除 (CascadeType.DELETE)** 概念：主實體被刪除時，關聯的實體也會自動一起被刪除，避免留下孤兒資料。
43. **級聯持久化 (CascadeType.PERSIST)** 概念：儲存主實體時，關聯的實體也會自動一起被存進資料庫，只需呼叫一次儲存方法。
44. **Hibernate 級聯操作類型 (續)** 概念：用表格整理 REMOVE、REFRESH、DETACH、MERGE、ALL 這幾種級聯類型分別做什麼，並示範 CascadeType.ALL 的寫法。
45. **Hibernate 級聯操作的配置細節** 概念：強調 Hibernate 預設不會做任何級聯，需要自己明確指定；也可以用逗號列出多種類型只做部分級聯。
46. **建立 Spring Boot 命令列應用程式 (Command Line App)** 概念：用 DAO 模式建一個命令列小工具，專門用來測試 JPA/Hibernate 的實作。
47. **第一步：定義 DAO 介面 (AppDAO Interface)** 概念：定義 AppDAO 介面跟一個 save 方法，先只定行為不寫實作。
48. **實作 AppDAO 實作類別 (AppDAO Implementation)** 概念：寫 AppDAOImpl，用建構子注入拿到 EntityManager，並用 entityManager.persist() 實作儲存，順便說明設了 CascadeType.ALL 後存主實體就會自動存到關聯實體。
49. **在應用程式中整合 AppDAO 與實體關聯** 概念：在 CommandLineRunner 裡注入 AppDAO，寫一個 createInstructor 方法把建立物件、建立關聯、呼叫存檔三個步驟串起來。
50. **實作一對一關係的物件關聯與儲存** 概念：示範用 setInstructorDetail() 把兩個物件連起來後，只呼叫一次 appDAO.save() 就能靠級聯把兩筆資料一起存進去。
51. **一對一單向關聯 (One-to-One Unidirectional) 的實作細節** 概念：解釋「擁有方 (Owning Side)」的概念——Instructor 是擁有方，所以外鍵欄位放在 instructor 表裡。
52. **執行資料庫腳本以建立資料表** 概念：實際跑 SQL 腳本建出 instructor 跟 instructor_detail 兩張表，並設好外鍵關聯。
53. **實作資源檔案概覽** 概念：說明下載的範例壓縮檔裡有 one-to-one、one-to-many、many-to-many 三種關聯的範例資料夾。
54. **建立進階 JPA 映射專案目錄** 概念：在專案資料夾下新建一個「09 Spring Boot JPA Advanced Mappings」目錄，把進階內容跟之前的基礎內容分開放。

55. **準備實作資源與環境** 概念：把起始檔案搬進新目錄，並開啟 MySQL Workbench 準備跑 SQL 腳本。
56. **初始化資料庫架構 (Schema)** 概念：找到 createDB.SQL 腳本，說明 Schema 就是一組資料表的集合，這個腳本會建出 HB01_1_to_1_Uni 這個 Schema。
57. **實作 SQL 腳本：重建資料表結構** 概念：腳本會先關掉外鍵檢查、刪除舊表，再重新建立 instructor_detail 跟 instructor 兩張表。
58. **instructor 資料表結構詳解** 概念：詳細列出 instructor 表的每個欄位定義，並用 CONSTRAINT 語法把 instructor_detail_id 設成外鍵。
59. **執行 SQL 腳本與結果驗證** 概念：在 MySQL Workbench 按下閃電圖示執行腳本，教你怎麼看綠色勾、黃色警告、紅色錯誤這些執行結果訊息。
60. **Schema 的進階概念與操作** 概念：提醒執行完要手動 Refresh 才看得到新 Schema，並補充 Schema 其實還包含 Views、預存程序、函數，不是只有資料表。
61. **使用 MySQL Workbench 進行反向工程 (Reverse Engineer)** 概念：教你怎麼用 Workbench 把現有資料庫結構自動轉成視覺化的 ER 圖表。
62. **完成反向工程與圖表檢查** 概念：說明產生圖表時要記得勾選匯入資料表物件、把物件放進畫布，並提醒圖表判讀的關聯類型有時會顯示錯誤，但只是視覺問題不影響實際結構。
63. **手動修正圖表中的關聯基數 (Cardinality)** 概念：教你在圖表裡點連線、選 Edit Relationship，手動把關聯基數改成 one to one。
64. **手動修正圖表關聯基數 (Cardinality)** 概念：重複整理同一個手動修正步驟，並再次強調圖表修改只是視覺呈現，真正決定關聯行為的還是程式碼裡的 @OneToOne 註解。
65. **使用 Spring Initializr 建立 Spring Boot 專案** 概念：到 start.spring.io 設定 Maven、Java、專案 group/artifact 等基本資訊，準備生成新專案。
66. **使用 Spring Initializr 建立 Spring Boot 專案 (續)** 概念：接著設定專案描述、package name，並加入 MySQL Driver 與 Spring Data JPA 這兩個依賴，最後按 GENERATE 下載專案。
67. **專案檔案準備與配置** 概念：把下載的 cruddemo.zip 解壓縮，搬到 DevSpringBoot 底下的 09 資料夾。
68. **開啟專案至 IntelliJ IDEA** 概念：用 IntelliJ 開啟專案，準備開始寫命令列應用程式。
69. **建立 Command Line 應用程式 (Command Line App)** 概念：介紹 CommandLineRunner 這個介面——等所有 Spring Bean 都載入完成後，裡面定義的方法就會自動執行。
70. **使用 Java Lambda 表達式實作 CommandLineRunner** 概念：用 Lambda 簡化寫法實作 CommandLineRunner，目前先印出 Hello World 當測試。
71. **基礎架構與框架準備** 概念：收尾說明目前只是把基礎設施搭好，後續會在這個基礎上寫更複雜的商業邏輯。

1. JPA / Hibernate 進階映射

現實世界的資料庫幾乎不會只有一張表，通常是好幾張表互相牽連，像一個大家庭一樣有各種親戚關係。前面學的「基礎映射」只是把一個 Java Class 對到一個資料表，像是一個人對一張身分證；而「進階映射」要處理的是這

些身分證之間的親屬關係——誰是誰的誰,這就是接下來要學的重點。

2. 一對一映射 (One-to-One Mapping)

想像講師(Instructor)本人是一張「基本資料卡」,而他還有一份「個人檔案」(Instructor Detail),裡面記錄 YouTube 頻道、興趣嗜好這些比較細節、不常用到的資訊。這種「一個人對一份檔案」的關係就是一對一。資料庫設計上會拆成兩張表:

- **Instructor 表**: `id`、`first_name`、`last_name`、`email`、`instructor_detail_id`(用來連到詳細資料表)
- **Instructor_detail 表**: `id`、`youtube_channel`、`hobby`

為什麼要拆兩張表而不是全部塞進一張?因為詳細資料通常不是每次查詢都需要,拆開可以讓核心資料表保持精簡,需要細節時再透過關聯去撈。

3. 一對多映射 (One-to-Many Mapping)

一名講師可以開很多門課,這就是「一對多」——一個講師對應到「一堆」課程。這裡先簡化假設一門課只會有一個講師開設(不考慮多位講師合開一門課的情況),把問題單純化方便理解概念。

4. 多對一映射 (Many-to-One Mapping)

多對一其實就是一對多「反過來看」而已。站在課程的角度看,很多門課程都指向同一位講師,這就是多對一。同一件事情,從講師的角度看是一對多,從課程的角度看就是多對一,只是觀察的立場不同。

5. 多對多映射 (Many-to-Many Mapping)

當兩邊都可以有「很多個」的時候就是多對多。經典例子是學生跟課程:一門課可以有很多學生選修,一個學生也可以同時選很多門課。雙方互相交叉配對,關係最複雜,後面會需要一張中介表來記錄誰選了誰。

6. 資料庫關聯性基礎概念

在真的動手寫 JPA 映射之前,要先把資料庫怎麼「定義關係」的底層機制搞懂,不然後面的註解只是死背。三個核心角色:

- **主鍵 (Primary Key)**: 每一列資料的身分證字號,獨一無二,用來認出「就是這一筆」。
- **外鍵 (Foreign Key)**: 一張表裡放著「別人家主鍵」的欄位,靠它把兩張表連起來,像是通訊錄裡存了朋友的身分證字號一樣。
- **級聯操作 (Cascading)**: 規定「動了父親,要不要順便動小孩」的連動規則,例如刪掉父實體時子實體要不要一起刪。

7. 資料庫關聯性核心要素

跟上一節講的是同一批觀念,再重複強調一次記憶點:主鍵負責「唯一識別」,外鍵負責「指向別人」。例如 `instructor` 表可以透過外鍵去關聯到 `instructor_detail` 表,外鍵欄位裡存的值,其實就是另一張表主鍵欄位的值。級聯操作則是定義「一個實體變了,關聯的實體要不要跟著變」。

8. 外鍵的實務應用範例

延續 Instructor 與 Instructor Detail 的例子:在 `instructor` 表裡設一個 `instructor_detail` 欄位當外鍵,它的值必須對應到 `instructor_detail` 表裡真實存在的主鍵。舉例來說,講師 Darby 的

`instructor_detail_id` 設成 100,這串數字就像一把鑰匙,直接開到 `instructor_detail` 表裡 `id = 100` 的那一筆資料,這樣就把兩張獨立的表串成一對一的關係了。

9. 資料庫級聯操作 (Cascading)

級聯講白了就是「幫主管簽了一份文件,底下附屬的文件也自動一起被簽了」——對父實體做一個操作,系統自動把同一個操作套用到所有關聯的子實體上。常見兩種:

- **儲存級聯 (Save Cascade):**存一個 `Instructor` 的時候,如果他關聯的 `Instructor Detail` 也是新資料,系統會順便一起存進資料庫,不用你分兩次手動存。
- **刪除級聯 (Delete Cascade):**刪掉一個實體時,自動把跟它綁在一起的其他實體也刪掉。為什麼要這樣? 因為講師都不存在了,他的詳細資料留著也沒意義,變成「孤兒資料」占空間又沒用,不如一起清掉。

10. 刪除級聯 (Cascading Delete) 的運作實例

實際運作起來就是:刪除講師 `Darby` 時,系統會自動去找跟 `Darby` 綁定的那筆 `instructor_detail`,一併刪除,不留殘骸。但這裡要提醒一句:刪除級聯不是「預設都要開」的功能,要看業務情境。像學生跟課程這種多對多關係,如果隨便開刪除級聯,刪一個學生可能連課程本身都被牽連刪掉,這就闖禍了——課程是獨立存在的,不該因為一個學生離開就消失。

11. 級聯操作的決策邏輯

這節其實是把上一節的警示講得更明白:級聯要不要開、開哪一種,完全取決於業務案例,不是「圖開發方便」就隨便加。用學生退選課程當例子:如果一名學生被刪除,正確做法是把他從課程名單裡「移除關聯」(退選),而不是把整門課程刪掉——課程還要留給別的學生用。開發者手上握有很細的控制權,可以針對每一種關聯分別決定要不要級聯,這是設計時要謹慎拿捏的地方,而不是無腦全開。

12. 資料檢索策略預覽

當我們把一個實體從資料庫撈出來時,關聯的資料要不要順便一起撈?這就衍生出兩種載入策略要學:及時載入 (Eager Loading) 跟延遲載入 (Lazy Loading),下一節細講。

13. 及時載入 (Eager Loading) 與 延遲載入 (Lazy Loading) 詳解

這兩個策略可以用「叫外送」來比喻:

- **及時載入 (Eager Loading):**就像叫外送時直接要求「主餐加所有配菜一起送到」,一次查詢就把主實體跟關聯資料全部抓進記憶體,之後要用隨時都在,但如果配菜根本用不到就白抓了。
- **延遲載入 (Lazy Loading):**先只送主餐,等你真的伸手去拿配菜(也就是程式碼真的存取到關聯欄位)時,才臨時再去廚房(資料庫)追加一次。這樣可以省下不必要的流量跟記憶體,是「按需取用」的精神。

14. 單向關聯 (Unidirectional Relationship) 概念預覽

單向關聯就是關係只有一個方向可以走,像單行道。以 `Instructor` 跟 `Instructor Detail` 為例,你可以從 `Instructor` 物件一路走到它的 `Instructor Detail`,但反過來——從 `Instructor Detail` 物件——你摸不到對應的 `Instructor`,因為 `Instructor Detail` 本身根本沒有存指向 `Instructor` 的引用。

15. 單向與雙向關聯 (Unidirectional vs. Bidirectional)

- **單向關聯 (Unidirectional)**:像剛剛講的,只能從 `Instructor` 找到 `Instructor Detail`,不能反過來。
- **雙向關聯 (Bidirectional)**:兩邊都能互相找到對方,從 `Instructor` 能拿到 `Instructor Detail`,從 `Instructor Detail` 也能直接拿到它所屬的 `Instructor`,像雙向道路一樣兩邊都能通行。

這裡有個重要心態:JPA/Hibernate 處理關聯沒有「唯一正解」,單向、雙向都是合法選擇,要看你的系統實際上需不需要「反查」這個功能。

16. 資料建模的靈活性與適應性

一對一、一對多、多對一、多對多這些模式,都只是「常見的建模方式」,不是死規定。課程裡教的範例應該當成通用指南來參考,實際專案要照著自己的功能需求跟業務邏輯去微調、甚至重新設計關聯方式,不用完全照抄教學案例。

17. Hibernate 一對一映射 (One-to-One Mapping)

一對一講的是兩個實體的實例彼此「唯一對應」——一個講師只有一份個人檔案,一份個人檔案也只屬於一位講師,不會共用或重複。在資料庫層級,這種關係還是要透過兩張獨立的表來實作(不是合併成一張),讓後面的映射範例都圍繞著 `Instructor / Instructor Detail` 這對組合展開。

18. 單向一對一關聯 (Unidirectional One-to-One)

這個範例先從最簡單的「單向」開始練習:關係只從 `Instructor` 指向 `Instructor Detail`,可以從講師物件拿到詳細資料,但無法反過來從詳細資料物件找回講師。先把單向搞熟,是之後學雙向關聯的墊腳石。

19. 一對一映射 (One-to-One Mapping) 的開發流程

實作一對一關聯建議照著這個順序走,不要跳步驟,不然容易卡關:

1. **資料庫準備階段 (Prep Work)**:先把資料表定義出來,並設好外鍵關聯。
2. **實體類別開發 (Entity Class Creation)**:先建立「被關聯」的子實體類別(例如 `InstructorDetail`),再建立主實體類別(例如 `Instructor`)。
3. **應用程式整合 (Application Integration)**:寫主程式,把前面兩塊組件串起來實際跑跑看。

這個順序背後的邏輯是「先有資料庫骨架,再有程式碼血肉」,資料庫沒設計好,Java 類別的映射註解就無從對應起。

20. 初始資料庫設計: `Instructor Detail` 表

動手寫 Java 類別之前,要先想清楚資料表長什麼樣子。`Instructor Detail` 表最基本的就是要有一個 `id` 當主鍵——這是所有表設計的起手式,先確保每一筆資料都能被唯一辨識。

21. 實作 `instructor_detail` 資料表腳本

把上一節的設計具體落成 SQL。`id` 設成主鍵並開自動遞增(不用自己手動編號,資料庫幫你顧),另外兩個欄位分別存 YouTube 頻道跟興趣:

```
CREATE TABLE instructor_detail (  
  id INT NOT NULL AUTO_INCREMENT,  
  youtube_channel VARCHAR(255),
```

```
hobby VARCHAR(255),  
PRIMARY KEY (id)  
);
```

22. instructor 資料表結構初步規劃

接著換規劃主表 `instructor`, 這節只是先起個頭, 列出目前確定要有的欄位: `id` 當主鍵、`first_name` 存名字, 其餘欄位下一節補完整。

23. instructor 資料表完整結構規劃

把 `instructor` 表的欄位補齊: `id`(主鍵、自動遞增)、`first_name`、`last_name`、`email`, 再加上一個關鍵的關聯預留欄位 `instructor_detail_id`, 用來指向 `instructor_detail` 的主鍵。

要注意的是, 到這個階段為止, `instructor` 跟 `instructor_detail` 在資料庫眼裡還是兩張完全獨立的表。雖然多了 `instructor_detail_id` 這個「把手」放在那邊, 但資料庫還沒有正式承認它們之間有關聯(還沒設外鍵約束), 下一步才要正式把它們接上線。

24. 外鍵 (Foreign Key) 的概念與作用

外鍵說穿了就是「A 表裡有個欄位, 裡面存的是 B 表的主鍵值」。它的作用就是把兩張原本毫無關係的表, 用這個欄位牽起紅線連在一起。

25. 實作一對一關聯：以 Instructor 為例

把外鍵概念實際套用到 `Instructor` 案例上, 分三步理解:

1. **建立連結點**: `instructor` 表裡的 `instructor_detail_id` 欄位就是外鍵本人。
2. **引用機制**: 這個欄位的值會指向 `instructor_detail` 表的主鍵。
3. **運作流程**: 要查某位講師的詳細資料時, 系統就拿 `instructor` 表裡這個外鍵值, 直接對到 `instructor_detail` 表裡對應的那一筆——就像拿著門牌號碼去找對應的房子。

26. 在 SQL 中實作外鍵約束 (Foreign Key Constraint)

光是欄位裡存一個數字, 資料庫並不會自動知道「這是外鍵、要拿去對應另一張表」, 必須用 `CONSTRAINT` 語法正式宣告這層關係, 包括取一個約束名稱、指定它是 `FOREIGN KEY`、指定本地欄位、指定要參照哪張表的哪個欄位:

```
-- 範例語法結構  
CONSTRAINT fk_instructor_detail  
FOREIGN KEY (instructor_detail_id)  
REFERENCES instructor_detail(id)
```

27. 參照完整性 (Referential Integrity)

定義外鍵不只是「接個線」而已, 更重要的目的是維持參照完整性——確保表跟表之間的關係不會被破壞。有了外鍵約束, 資料庫會主動擋下兩種危險操作:

- **孤立紀錄**: 不准在 `instructor` 表塞一個指向根本不存在的 `instructor_detail_id`。

- **無效刪除**:不准隨便刪掉一筆還被別人引用著的 `instructor_detail` 紀錄(除非有設級聯)。

一句話總結:「如果 A 說自己引用了 B,那 B 就必須真的存在」,這條規則由資料庫層級強制把關,不是靠程式自律。

28. 外鍵約束的防禦機制

延續上一節,外鍵約束其實就是資料庫的「品管關卡」:外鍵欄位裡的值,一定要是另一張表裡真實存在的主鍵,否則資料庫會直接丟錯誤擋下來,不會讓壞資料混進去。這也代表開發者寫程式時要遵守這套參照規則,別想著繞過去,不然資料庫的一致性會被破壞。

29. 實作進度總結與展望

盤點一下目前做完的事:資料庫表結構設計好了,外鍵關聯也定義完成了。接下來要從「資料庫層級」的思維切換到「Java 程式碼層級」,準備開始學怎麼用 JPA/Hibernate 把這些關聯寫進 Java 類別裡,完成真正的一對一映射功能。

30. 進入 Java 實體類別開發階段

回顧一下開發流程(對照第 19 節的三大階段):第一階段的資料庫準備工作已經做完,接下來要進入第二、三階段——建立實體類別、整合應用程式。第一個任務是實作 `InstructorDetail` 類別,目標是讓這個 Java 類別跟資料庫裡已經存在的 `instructor_detail` 表對應起來。實體類別的核心邏輯,就是讓它在程式碼裡「代表」資料庫的一筆紀錄,讓開發者可以用物件導向的方式操作資料,而不用直接寫 SQL。

31. 實作 `InstructorDetail` 實體類別映射

寫 Java 類別時要靠 JPA 註解告訴 Hibernate「這個類別對應哪張表、哪些欄位對應哪些屬性」:

- `@Entity`:貼上這個標籤,代表這個類別正式成為受 JPA 管理的實體。
- `@Table(name = "instructor_detail")`:明確指定要映射到哪張表,不寫的話 Hibernate 會用類別名稱去猜。
- 主鍵欄位(`id`)跟一般欄位(`youtubeChannel`、`hobby`)都要各自映射到資料庫對應欄位。
- 除了映射註解,類別裡照樣要有標準的建構函式、Getter/Setter,這些是 Java Bean 的基本禮貌,少了 Hibernate 會不認得怎麼操作物件。

32. 實作 `Instructor` 實體類別映射

子實體做完後,換主實體 `Instructor` 上場,一樣用 `@Entity` 加 `@Table(name = "instructor")` 宣告映射關係。本質上跟上一節做的事情一樣:定義「Java 物件」跟「資料庫紀錄」的對應表,讓程式能用物件的方式操作 `instructor` 表的資料。

33. 建立實體類別間的關聯映射

到這裡,`Instructor` 跟 `InstructorDetail` 各自的欄位(姓名、email 等)雖然都映射好了,但兩個 Java 類別彼此之間仍然是互不相干的陌生人——就算資料庫裡已經用外鍵接起來了,Java 程式碼還完全不知道這件事。

解法是在 `Instructor` 類別裡加一個新屬性,並貼上 `@OneToOne` 註解,靠這個註解才能讓 Hibernate 知道「這兩個類別之間有一對一關係」,把資料庫裡的外鍵關係「翻譯」進 Java 世界。

34. 實作關聯的連結機制 (Hooking up the Relationship)

只貼 `@OneToOne` 還不夠,那只是宣告「有關係」,但沒講清楚「靠哪個欄位對接」。這時候要靠 `@JoinColumn` 註解,在 `Instructor` 類別的關聯屬性上指定實際負責存外鍵值的欄位名稱(例如 `instructor_detail_id`)。

有了這個設定,Hibernate 背後的自動化流程是這樣運作的:

1. **識別外鍵**:讀取 `@JoinColumn` 指定欄位的值。
2. **執行查詢**:拿這個外鍵值當條件,自動跑去 `instructor_detail` 表撈對應的那筆紀錄。
3. **物件組裝**:把撈到的資料組裝成 `InstructorDetail` 物件,直接掛在 `Instructor` 物件上——整個過程開發者完全不用手動寫查詢,拿到手的 `Instructor` 物件裡就已經包好了它的 `InstructorDetail`。

35. Hibernate 實體生命週期 (Entity Life Cycle)

在學級聯類型 (Cascade Types) 的細節之前,要先搞懂一件更基礎的事:一個 Hibernate 實體在程式運行期間會經歷哪些「狀態」。這些狀態決定了 Hibernate 到底有沒有在「盯著」這個物件的變化。

這節先介紹**游離狀態 (Detached State)**:當一個實體處於游離狀態,代表它已經跟目前的 Hibernate Session(可以想成一次「工作階段」)脫鉤了。物件本身還好好地活在 Java 記憶體裡,但 Hibernate 已經不再監控它的任何變動——這時候你改了物件的屬性值,那個改動不會自動同步進資料庫,除非你想辦法把它重新掛回 Session 上。這概念很重要,因為很多「明明改了資料但存不進去」的 bug,根源就是物件已經處於游離狀態卻不自知。

36. Hibernate 實體狀態操作方法

知道有哪些狀態還不夠,還要知道怎麼「切換」這些狀態,靠的是 Session 上的幾個方法。可以把 Hibernate Session 想像成一個「戶籍事務所」,這幾個方法就是不同的窗口服務:

- **merge (重新關聯)**:專門處理**游離 (Detached)** 狀態的實體——物件已經脫離戶籍了,拿去這個窗口重新「合併」登記,讓它變回受管理 (Managed) 狀態。
- **persist (儲存新實體)**:專門處理全新的實體,把它從「暫時」狀態轉成受管理狀態,等下一次 `flush` 或 `commit` 時,Hibernate 就會把它正式寫進資料庫,像新生兒辦出生登記。
- **remove (刪除)**:把一個受管理狀態的實體標記為要刪除,下一次 `flush` 或 `commit` 時會真的從資料庫移除,像辦除戶。
- **refresh (重新整理/同步)**:強制把記憶體中的物件內容,跟資料庫目前實際的資料重新對齊。為什麼需要它?因為如果資料庫裡的資料已經被別的程序改過,但你手上的物件還抱著舊資料不放,這就是所謂的**陳舊資料 (Stale Data)**——`refresh` 就是用來刷新這份「過期戶籍謄本」,確保手上的資料是最新版本。

37. Hibernate 實體狀態轉換流程圖

這節其實只是個「開場白」,講師說接下來會用一張流程圖來講解物件在生命週期裡怎麼流動。如果你是那種看文字容易恍神、看圖才會懂的人,這張圖會比較有感。重點只有一句:物件的狀態轉換用「圖」來理解會比死記文字規則更直觀。

38. 狀態與操作細節說明

這節把 Hibernate 實體的三種主要狀態講清楚,可以想像成一個人「入職」的過程:

- **Transient (暫時 / New)**:剛用 `new` 生出來的物件,跟資料庫完全沒關係,就像一份履歷還沒投出去。呼叫 `persist` 或 `save` 之後才會「轉正」進入 Persistent。
- **Persistent (持久化 / Managed)**:這是「已經被 Hibernate Session 管理」的狀態,好比正式員工,Session 會盯著它的一舉一動。這個狀態下常見的操作有:

- **refresh**：把記憶體裡的物件跟資料庫最新資料同步（怕資料庫被別人改過）。
- **commit**：把變更真的寫進資料庫,交易正式生效。
- **rollback**：反悔，把這次交易的變更全部撤銷。
- **Detached (游離)**：原本是 Managed，但 Session 被關掉 (**close**) 後就變成「離職」狀態，物件還在，但沒人管它了。之後可以用 **merge** 讓它「回鍋」變回 Persistent，或者就放著不管，讓它退回 Transient。

39. Hibernate 狀態轉換的進階細節

這節補充兩個容易搞混的轉場動作：

- **remove (刪除)** 之後：對一個 Managed 物件呼叫 **remove**，它會先進入 **Removed (已標記刪除)** 狀態——注意是「標記」，還沒真的刪。等到 **commit** 執行後，資料庫那筆紀錄才會真的被砍掉，這時物件的狀態變回 **Transient**（因為它已經不對應資料庫裡任何東西了）。
- **rollback** 的影響：不管你原本是 Managed 還是 Removed，只要 **rollback**，都會被打回 **Detached** 狀態，跟目前這次交易斷開關係。

學習建議也很實在：**不要硬背所有狀態轉換的細節**，Hibernate 的狀態機邊緣案例一大堆，真正要抓的是核心概念——「物件跟 Session 的關聯性」以及「記憶體資料跟資料庫資料何時同步」，其他瑣碎路徑用到再查就好。

40. Hibernate 級聯操作 (Cascading) 概念

Cascading (級聯) 講白話一點就是「一人做事一人擔，但這裡是一個操作波及一群人」。你對主實體做的動作（例如 save），會自動傳導到跟它關聯的實體上，不用你一個一個手動處理。

舉例：**Instructor** (講師) 跟 **InstructorDetail** (講師詳細資料) 是關聯的兩個實體。如果設定了級聯，你只要 **save** 講師，Hibernate 就會順便把講師的詳細資料也存進資料庫，你不用另外再呼叫一次 save。

這種設計特別適合「附屬關係」很明確的情境——沒有講師，講師詳細資料根本不該存在，兩者生命週期綁在一起才合理。

41. Hibernate 級聯操作類型 (Cascade Types)

級聯不是一個「全開或全關」的開關，而是可以像自助餐一樣挑選你要哪幾種行為套用到關聯實體上，這就是 **CascadeType**。它讓你精細控制「主實體的哪些操作」該傳給關聯實體，哪些不該傳。

42. 級聯刪除 (CascadeType.DELETE)

主實體被刪掉時，Hibernate 會自動把關聯的實體紀錄也一併刪除。以 **Instructor** 跟 **InstructorDetail** 設定 **CascadeType.DELETE** 為例：刪除某位講師時，對應的講師詳細資料會自動從資料庫消失。這樣做的目的是避免資料庫裡出現「孤兒資料」——也就是那種指向已經不存在的主實體、卻還留在資料庫裡沒人理的殘留紀錄。

43. 級聯持久化 (CascadeType.PERSIST)

主實體進入 Persistent 狀態時，關聯實體會跟著一起進入該狀態。也就是說儲存 **Instructor** 的時候，如果設了 **CascadeType.PERSIST**，Hibernate 會自動把關聯的 **InstructorDetail** 也存進資料庫。好處很直接：開發者只需要呼叫一次「存主實體」的方法，整組關聯物件就都存好了，不用自己寫兩次 save。

44. Hibernate 級聯操作類型 (續)

除了 DELETE 跟 PERSIST，還有其他幾種級聯類型，整理成表格比較好記：

Cascade Type	說明
REMOVE	主實體被移除/刪除，關聯實體也跟著被刪除
REFRESH	主實體同步 (Sync) 時，關聯實體也一起從資料庫更新
DETACH	主實體變成游離狀態 (跟 Session 斷開)，關聯實體也跟著游離
MERGE	主實體合併 (Merge) 時，關聯實體也一起合併
ALL	把上面所有級聯類型全部打包，一次全開

如果你懶得一個一個挑，想要「主實體做什麼，關聯實體就跟著做什麼」，直接用 **ALL** 最省事：

```
@OneToOne(cascade = CascadeType.ALL)
private InstructorDetail instructorDetail;
```

45. Hibernate 級聯操作的配置細節

有兩個實務上很重要的細節：

- **預設是「不級聯」**：如果你在 **@OneToOne** (或其他關聯註解) 裡完全沒寫 **cascade** 屬性，Hibernate 不會幫你做任何級聯，什麼都要自己手動處理。這點很容易被新手忽略，以為關聯設好了操作就會自動連動，結果測試時才發現關聯物件根本沒存進去。
- **可以只挑幾種類型組合**：不一定要用 **ALL**，如果只想要「持久化」跟「刪除」會連動，但不想要合併或重新整理也連動，可以用逗號分隔的清單精細控制：

```
// 同時套用 PERSIST 與 REMOVE，但不包含其他類型
@OneToOne(cascade = {CascadeType.PERSIST, CascadeType.REMOVE})
private InstructorDetail instructorDetail;
```

46. 建立 Spring Boot 命令列應用程式 (Command Line App)

這節開始進入實作階段。目標是做一個專門的命令列小工具，用來練習 JPA、Hibernate 跟 DAO 模式，不需要真的做網頁介面，專心測資料庫邏輯就好。

架構延續之前教過的 **DAO (Data Access Object)** 模式，把「業務邏輯」跟「資料庫存取邏輯」分開兩層：

- **Main Application**：程式的進入點，負責跑業務邏輯。
- **AppDAO**：負責跟資料庫溝通的那一層，Main Application 要存東西都得透過它。

47. 第一步：定義 DAO 介面 (AppDAO Interface)

先定義「該做什麼事」，不管「怎麼做」——這是介面 (interface) 的精神。建立一個 **AppDAO** 介面，裡面先放一個核心方法：

```
public interface AppDAO {  
    public void save(Instructor theInstructor);  
}
```

`save` 方法接收一個 `Instructor` 物件，目標是把這位講師連同他所有關聯的組件（例如 `InstructorDetail`）一起存進資料庫。注意這裡只是定義介面，實作留到下一節。

48. 實作 AppDAO 實作類別 (AppDAO Implementation)

現在來寫真正做事的 `AppDAOImpl` 類別，實作前面定義的 `AppDAO` 介面。核心是拿到一個 `EntityManager`——這是 JPA 世界裡跟資料庫溝通的主要工具，可以想成一個「萬用資料庫操作窗口」。

取得方式建議用**建構子注入 (Constructor Injection)**，而不是欄位注入，這樣可以確保物件一被建出來，依賴的東西就已經準備好了，是比較穩健的寫法。

`save` 方法的實作很簡單，就一行：`entityManager.persist(theInstructor)`。

但關鍵在後面——因為 `Instructor` 實體上已經設了 `CascadeType.ALL`，所以呼叫這一行 `persist` 時，Hibernate 不只存 `Instructor` 本身，還會自動偵測到關聯的 `InstructorDetail`，順便幫你把它也 `persist` 存進資料庫。這就是級聯真正發威的地方——你寫一行程式碼，Hibernate 幫你做兩件事。

49. 在應用程式中整合 AppDAO 與實體關聯

這節把前面的 DAO 接到 Main Application 裡實際跑起來。步驟是：

- 在 `CommandLineRunner` 裡注入 `AppDAO`，這樣程式啟動時就能執行業務邏輯。
- 寫一個輔助方法（例如叫 `createInstructor`）把「建立實體 + 建立關聯」這段稍微複雜的流程包起來，讓主邏輯乾淨一點。

實體建立與關聯的流程分三步：

1. **建立實體物件**：`new Instructor()` 跟 `new InstructorDetail()`，兩個都還是 `Transient` 狀態的空殼。
2. **建立關聯**：用類似 `setInstructorDetail()` 的方法把兩個物件接在一起。這步一定要在 `persist` 之前做完，不然 Hibernate 根本不知道這兩個物件有關聯，級聯也就無從發生。這是最容易踩的坑：忘記先關聯就直接 `save`，結果 `InstructorDetail` 沒被存進去。
3. **執行持久化**：呼叫 `appDAO.save(theInstructor)`。因為已經設了 `CascadeType.ALL`，只要對主實體 (`Instructor`) 做一次 `persist`，Hibernate 就會自動把關聯的 `InstructorDetail` 一起處理掉。

50. 實作一對一關係的物件關聯與儲存

這節再強調一次「先關聯、再儲存」的順序，並補上程式碼範例：

```
// 將 instructorDetail 物件與 instructor 物件進行關聯  
tempInstructor.setInstructorDetail(tempInstructorDetail);
```


這一行就是建立一對一（One-to-One）關係的關鍵動作。沒做這步，Hibernate 完全不知道這兩個物件之間有任何瓜葛。

接著呼叫 `appDAO.save(tempInstructor)` 時，底層發生的事情是：`AppDAOImpl` 把這個請求丟給 `EntityManager.persist()`；因為設了 `CascadeType.ALL`，Hibernate 在持久化 `tempInstructor` 的同時，會自動偵測到它身上掛著的 `instructorDetail`，順便對這個關聯物件也做一次 `persist`。

好處就是：開發者不用分別對兩個物件呼叫儲存方法，只要處理主實體，整棵關聯物件樹就一起存好了，程式碼簡潔很多。

51. 一對一單向關聯 (One-to-One Unidirectional) 的實作細節

這節講的是「單向關聯」的資料庫結構意義。單向關聯的意思是只有一邊（例如 `Instructor`）拿著另一邊（`InstructorDetail`）的引用，反過來 `InstructorDetail` 並不知道有哪個 `Instructor` 指著它。這個關聯關係會直接反映在資料庫的表結構上。

這裡有個重要概念叫「擁有方（Owning Side）」：擁有方就是負責定義「物理映射關係」的那一方。在這個範例裡，`Instructor` 是擁有方，關鍵影響是：外鍵會存在 `instructor` 資料表裡，指向 `instructor_detail` 的主鍵——而不是反過來。這個「誰是擁有方、外鍵放哪邊」的判斷，之後常常會影響你怎麼寫 SQL 跟怎麼設計資料表。

最後提醒：關聯要用單向還是雙向，要看應用程式實際需求跟領域模型（Domain）來決定，沒有一定要套的公式。

52. 執行資料庫腳本以建立資料表

進入實作面，要建立兩張表：`instructor` 表和 `instructor_detail` 表，並且在 `instructor` 表裡設一個外鍵指向 `instructor_detail`（呼應上一節「擁有方在 `Instructor`」的結論）。

兩張表的欄位設計：

- `instructor` 表：`id` (INT)、`first_name` (VARCHAR(45))、`last_name` (VARCHAR(45))、`email` (VARCHAR(45))、`instructor_detail_id` (INT)——這個欄位就是用來存外鍵，指向詳細資訊表。
- `instructor_detail` 表：`id` (INT)、`youtube_channel` (VARCHAR(128))、`hobby` (VARCHAR(45))。

53. 實作資源檔案概覽

這節是操作提示，教你把課程提供的 ZIP 檔解壓縮出來看看有什麼。裡面會有幾個對應不同關聯類型的資料夾：`one-to-one`（一對一）、`one-to-many`（一對多）、`many-to-many`（多對多），這些會在後續影片依序拿來實作，這節只是先讓你知道資源長什麼樣子。

54. 建立進階 JPA 映射專案目錄

在 `Dev Spring Boot` 資料夾底下新建一個目錄叫 `09 Spring Boot JPA Advanced Mappings`，專門用來放進階映射主題的實作內容。純粹是為了把「進階內容」跟「之前的基礎內容」在檔案系統上分開，避免專案愈疊愈亂、找不到東西。

55. 準備實作資源與環境

把下載好的 **00 starter files** (起始程式碼) 移到剛剛建的 **09 Spring Boot JPA Advanced Mappings** 目錄裡，確保開發環境齊全。接著打開 **MySQL Workbench** 登入資料庫，準備下一步要開啟先前整理好的 SQL 腳本，來初始化資料庫結構。這節本身沒什麼技術含量，就是「開工前把工具跟檔案都擺好」的準備動作。

56. 初始化資料庫架構 (Schema)

打開目錄路徑 **09 Spring Boot JPA Advanced Mappings** → **00** → **HB01 1 to 1 uni**，選擇 **createDB.SQL** 這個檔案。

順便解釋一下 Schema 是什麼：**Schema** 本質上就是一組資料表的集合，可以想成是一個「資料庫裡的資料夾」，把相關的表都收在一起。這個範例的腳本會建立一個叫 **HB01_1_to_1_Uni** 的 Schema，之後 **instructor** 跟 **instructor_detail** 這兩張表都會放在裡面。

57. 實作 SQL 腳本：重建資料表結構

寫 SQL 腳本重建資料表時，有兩個預處理動作很重要，可以避免腳本跑到一半炸掉：

- **停用外鍵檢查**：在刪表或建表之前先關掉外鍵檢查 (disable foreign key checks)。原因是如果某張表還被別的表的外鍵參照著，直接刪除會觸發參照完整性錯誤，讓腳本中斷跑不完。
- **清理舊有結構**：用 **DROP TABLE IF EXISTS** 先把可能存在的舊表 (例如 **instructor_detail**) 刪掉，確保是乾淨的起始狀態，避免「表已存在」這種報錯。

建立 **instructor_detail** 表：**id** (INT) 設 **AUTO_INCREMENT** 當主鍵、**youtube_channel** (VARCHAR(128))、**hobby** (VARCHAR(45))。

建立 **instructor** 表則遵循一樣的流程——先檢查刪除舊表，再重新定義結構，並且要記得加上指向 **instructor_detail** 的外鍵欄位，這樣才能完成一對一的關聯映射 (外鍵放在 **instructor** 表，呼應第 51 節講的「擁有方」概念)。

58. instructor 資料表結構詳解

instructor 表的完整欄位：**id** (Auto Increment 主鍵)、**first_name**、**last_name**、**email**，以及關鍵的 **instructor_detail_id**——這是連到詳細資訊表的欄位。

外鍵約束的原理是：把 **instructor.instructor_detail_id** 這個欄位跟 **instructor_detail** 表的主鍵綁在一起，確保兩張表之間資料一致 (不會出現 **instructor_detail_id** 指向一個根本不存在的 id)。SQL 寫法：

```
CONSTRAINT fk_instructor_detail
  FOREIGN KEY (instructor_detail_id)
  REFERENCES instructor_detail(id)
```

59. 執行 SQL 腳本與結果驗證

在 MySQL Workbench 工具列點擊黃色閃電圖示執行腳本，然後看下方 **Output** 區域確認結果。三種顏色的意義：

- **綠色勾選**：這條指令成功了。

- **黃色警告**：通常可以無視，只要沒有紅色錯誤就代表表已經建好了。
- **紅色錯誤**：執行失敗，要修正 SQL 語法或結構問題後重跑。

這裡有個設計上的取舍值得注意：雖然 SQL 本身可以定義 **ON DELETE NO ACTION** 之類的級聯約束，但這套課程的做法是刻意不在 **SQL 腳本層級定義級聯行為**，而是把級聯邏輯全部交給應用程式層的 Hibernate 來管理（也就是前面第 40~45 節講的 **CascadeType**）。這樣做的好處是彈性更高，邏輯集中在 Java 程式碼裡管理，不用分散在資料庫跟程式兩邊。

60. Schema 的進階概念與操作

如果跑完 SQL 腳本後左側 Schema 列表沒看到新資料庫，記得點 **Refresh All** 重新整理一下，才會看到新建的 **HB01_1_to_1_Uni**。

順便補充一個容易被忽略的觀念：Schema 的完整定義不只是「一堆表」，廣義來說還包含 **Views（檢視表）**、**Stored Procedures（儲存程序）**、**Functions（函數）**，只是這門課主要專注在資料表上。實務上也可以把新建的 Schema 設成 **Default Schema**，之後查詢就不用每次都指定資料庫名稱。

要檢查表的狀態，可以右鍵點資料表選 **Select Rows** 看內容——這個階段因為還沒跑插入資料的腳本，**instructor** 等表應該都是空的，這是正常現象。

61. 使用 MySQL Workbench 進行反向工程 (Reverse Engineer)

反向工程 (Reverse Engineer) 的目的是把現有的資料庫結構（表、Schema）自動轉換成視覺化的 ER 圖表，讓你一眼看出實體之間的關聯，而不用自己土法煉鋼畫圖。

操作步驟：

1. 選單列選 **Database → Reverse Engineer**。
2. 選擇連線（例如 **local hbstudent**），點 **Continue**。
3. 勾選目標 Schema（例如 **hb-01-one-to-one-uni**），點 **OK**。
4. 系統自動讀取該 Schema 下所有資料表，畫出圖表。

62. 完成反向工程與圖表檢查

過程中要確保勾選兩個選項：**Import MySQL Table Objects**（把資料表物件匯入工具）跟 **Place Objects on a Diagram**（自動把物件排到圖表畫布上）。完成後依序點 **Execute → Continue → Close**，就能看到自動生成的圖表，例如 **instructor** 跟 **instructor_detail** 之間的關係圖。

有個常見的小狀況：MySQL Workbench 有時候沒辦法正確判讀關聯的精確類型，例如把「一對一」誤判成別的關係類型。不用緊張，這通常只是**視覺上的修飾問題（Cosmetic Issue）**，並不會影響資料庫實際的物理結構或約束條件——資料庫該有的外鍵約束都還是好好的，只是畫出來的圖標示錯了。

63. 手動修正圖表中的關聯基數 (Cardinality)

如果自動生成的圖表關聯類型判讀錯誤（比如一對一被畫成別的），可以手動修正讓圖表更好讀。步驟：

1. 在圖表中點選代表關係的連線。
2. 選 **Edit Relationship**。
3. 切到下方的 **Foreign Key** 頁籤，在 **Cardinality** 選項把它改成 **one to one**。
4. 設定完成後圖表會即時更新成正確的一對一呈現。

這裡的關鍵觀念要記住：圖表上的修正只是視覺修飾，實際的業務邏輯與關聯行為，還是得靠後面 Java 程式碼裡的 JPA/Hibernate 設定 (@OneToOne 註解) 來決定，不能只看圖表就以為系統行為是對的。

64. 手動修正圖表關聯基數 (Cardinality)

這節內容跟第 63 節幾乎重複，再講一次同一件事：如果自動生成的圖表沒辦法正確顯示「一對一」關係，可以手動編輯提升可讀性。操作步驟一樣是：選取連線 → **Edit Relationship** → 切到 **Foreign Key** 分頁 → 把 **Cardinality** 改成 **one-to-one**。

核心觀念也再強調一次：****視覺修飾 (Cosmetic) 跟邏輯定義 (Logical) ****是兩回事——Workbench 裡改的只是讓圖表看起來對，真正決定系統行為的是 Java 程式碼裡的 @OneToOne 註解。這兩節連續出現同樣的內容，可能是原始逐字稿的重複段落，但也剛好強化了這個「圖表只是輔助理解、程式碼才是真相」的重點。

65. 使用 Spring Initializr 建立 Spring Boot 專案

終於要開始建立真正的 Spring Boot 專案了。前往 start.spring.io (Spring Initializr 是官方提供的專案產生器，幫你將 Maven/Gradle 設定、依賴都打包好，省去手動設定 pom.xml 的麻煩)。

專案基本設定：

- **Project** : Maven
- **Language** : Java
- **Spring Boot** : 選最新的正式發佈版本 (Released version)，避免選帶 (SNAPSHOT) 字樣的——SNAPSHOT 是還在開發中、不穩定的版本，正式開發不建議用。

專案元數據 (Metadata) 設定：

- **Group** : com.love2code
- **Artifact** : cruddemo
- **Name** : cruddemo
- **Description** : Demo project for Spring Boot
- **Package name** : com.love2code.cruddemo
- **Packaging** : Jar
- **Java** : 25 (依畫面顯示的版本)

66. 使用 Spring Initializr 建立 Spring Boot 專案 (續)

接續上一節，把元數據再確認一次 (注意這裡 Package name 打成 com.luv2code.cruddemo，跟上一節的 com.love2code 拼法不太一樣，這是原始逐字稿的用字差異，實作時以講師實際畫面為準)：Artifact 是 cruddemo、Name 是 cruddemo、Description 是 Demo project for Spring Boot、Packaging 選 Jar、Configuration 選 Properties、Java 版本依環境選 (例如 25)。

重點是這節新增的 **Maven 依賴項 (Dependencies)**，要支援資料庫操作，需要加兩個：

- **MySQL Driver** : 提供 MySQL 的 JDBC 驅動程式支援，讓 Java 程式能連上 MySQL。
- **Spring Data JPA** : 讓你用 JPA (Java Persistence API) 搭配 Hibernate 來做資料持久化，這正是這整份筆記前面一直在講的機制的基礎依賴。

確認依賴跟設定都沒問題後，點 **GENERATE** 按鈕，專案檔案就會打包下載到本機。

67. 專案檔案準備與配置

把剛剛下載的 `cruddemo.zip` 從 `Downloads` 資料夾找出來，解壓縮，然後把解壓後的資料夾複製到指定的開發路徑：`DevSpringBoot` → `09` 資料夾裡。這節純粹是檔案整理的操作步驟，沒有技術概念，重點只是「把專案放到正確的位置，方便之後統一管理」。

68. 開啟專案至 IntelliJ IDEA

用 IntelliJ IDEA 開啟剛剛準備好的專案：導航到 `09-spring-boot-jpa-advanced-...` 目錄，開啟主要的 Spring Boot 應用程式檔案，準備要開始寫命令列應用程式（Command Line App）的程式碼。這節也是操作性質，為下一節的實際寫程式做準備。

69. 建立 Command Line 應用程式 (Command Line App)

介紹 **CommandLineRunner**——這是 Spring Boot 框架提供的一個介面，執行時機是：當所有 **Spring Beans** 都已經成功載入之後，這個介面定義的方法就會被自動觸發執行。可以把它想成「開店前的最後檢查清單跑完之後，自動開門營業」的那個動作，很適合用來寫應用程式啟動時要跑的初始化邏輯或命令列任務。

基本骨架：

```
@Bean
public CommandLineRunner commandLineRunner(String[] args) {
    return args -> {
        // 這裡撰寫要執行的邏輯
    };
}
```

70. 使用 Java Lambda 表達式實作 CommandLineRunner

上一節的骨架裡用的 `args -> { ... }` 其實就是 **Lambda 表達式**，是實作 `CommandLineRunner` 介面的簡寫語法（Shorthand notation）。用 Lambda 可以省去寫一整個匿名內部類別的樣板程式碼，讓邏輯更精簡好讀。

同樣要記得：這段程式碼一定是在所有 **Spring Beans** 都成功載入之後才會執行，這個時機點很重要，因為後面章節會需要用到已經注入好的 Bean（例如 AppDAO）來執行業務邏輯。

目前最簡單的實作範例：

```
@Bean
public CommandLineRunner commandLineRunner(String[] args) {
    return runner -> {
        System.out.println("Hello World");
    };
}
```

目前只是印出 `Hello World` 當作驗證程式能跑起來，後續章節才會真的利用載入好的 Spring Beans（例如前面做好的 AppDAO）去執行儲存講師資料之類的實際業務邏輯。

71. 基礎架構與框架準備

這節是收尾性質的過渡段落，講師點出目前的開發進度：現階段的重點都放在打好**基礎設施（Infrastructure）跟框架（Framework）**上——也就是專案結構、資料庫連線、DAO 介面、CommandLineRunner 骨架這些「地基工程」，把地基打穩之後，後面才會開始疊上真正的業務邏輯（像是儲存講師與講師詳細資料的完整流程）。